



Assignment

Institute Name:	Silver Oak College of Computer Application	Programme Name:	MCA/MSCIT/MSCDS
Semester:	I	Academic Year:	2026-27
Course Name:	Foundation of Mathematics	Course Code:	MDC5001C

Index of Assignments

No.	Assignment Title	Unit	CO	Bloom's Level
1	How Many Passwords Are Really Possible?	Unit 1: Introduction to Combinatorics and Its Applications	CO1	Apply
2	Cracking the Code: The Pigeonhole Principle in Hashing	Unit 1: Introduction to Combinatorics and Its Applications	CO1	Analyze
3	Catching the Copycats: Inclusion-Exclusion in Data Deduplication	Unit 1: Introduction to Combinatorics and Its Applications	CO1	Evaluate
4	Predicting Growth: Recurrence Relations Behind Recursive Algorithms	Unit 1: Introduction to Combinatorics and Its Applications	CO1	Create
5	Root of the Problem: Finding Break-Even Points	Unit 2: Numerical Methods	CO2	Apply
6	Solving the Network Load-Balancing Puzzle	Unit 2: Numerical Methods	CO2	Analyze
7	Predicting Missing Data Points: The Interpolation Detective	Unit 2: Numerical Methods	CO2	Apply
8	Which Method Wins? A Numerical Methods Showdown	Unit 2: Numerical Methods	CO2	Evaluate
9	Maximizing Profit: The Graphical LPP Challenge	Unit 3: Operations Research and Linear Programming	CO3	Apply
10	From Warehouse to Customer: Solving the Transportation Puzzle	Unit 3: Operations Research and Linear Programming	CO3	Analyze
11	Who Does What Best? The Assignment Problem	Unit 3: Operations Research and Linear Programming	CO3	Analyze
12	Game On: Strategic Decisions Using Game Theory	Unit 3: Operations Research and Linear Programming	CO3	Evaluate
13	Ordering the Chaos: POSETs in File Permission Systems	Unit 4: Lattices and Boolean Algebra	CO4	Apply



No.	Assignment Title	Unit	CO	Bloom's Level
14	Designing Digital Logic: Boolean Algebra in Circuit Design	Unit 4: Lattices and Boolean Algebra	CO4	Analyze
15	The Lattice Behind Your Access Control System	Unit 4: Lattices and Boolean Algebra	CO4	Create

Unit 1: Introduction to Combinatorics and Its Applications

Assignment Title: How Many Passwords Are Really Possible?

Assignment Type: Real-Life Investigation

CO: CO1 – Apply combinatorial techniques to solve discrete mathematical problems in computer science.

Bloom's Taxonomy Level: Apply

Scenario / Context

Every password policy — minimum length, allowed characters, whether repetition is permitted — silently defines a “keyspace”: the total number of passwords an attacker would have to try. Counting principles (permutations and combinations, with and without repetition) let you calculate this keyspace exactly, and from it, estimate how long a brute-force attack would realistically take.

Student Task

You will analyze the password policies of three real platforms/apps using counting principles, calculate their total keyspace, and estimate brute-force crack times.

Instructions / Activity Steps

1. Note the password policy of 3 real platforms you use (e.g., email, banking app, university portal) — minimum/maximum length and allowed character sets (digits, uppercase, lowercase, symbols).
2. For each policy, calculate the total number of possible passwords assuming characters may repeat (permutations with repetition).
3. Recalculate assuming no character may repeat (permutations without repetition), and note any practical limitations this creates.
4. Assuming a brute-force attack rate (e.g., 10^9 guesses/second), calculate the worst-case time needed to crack each policy.
5. Compare the three policies and rank them from most to least secure based on your calculations.

Questions for Analysis

1. Which policy has the largest keyspace, and by how many orders of magnitude does it exceed the smallest?
2. How does adding just one more allowed character to the set affect the total number of combinations — is the growth linear or exponential? Explain using the counting formula.
3. Why do many real systems disallow certain “predictable” passwords even though they fall within



the counted keyspace?

Think Beyond the Formula

- If a platform allowed emojis or Unicode characters in passwords, how would that affect keyspace size and real-world security?

Expected Submission

- Table of the 3 policies with keyspace calculations
- Brute-force crack-time estimates for each
- Ranking of the 3 policies with justification
- One-paragraph reflection

ICT Tool / Resources Required

Scientific calculator; MS Excel for large-number computation

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of counting principles (permutations, combinations, with/without repetition)	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct calculation of keyspace for all 3 policies	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate brute-force time estimation and ranking logic	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Presentation / clarity of tables	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original reflection connecting to real password security	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10



Unit 1: Introduction to Combinatorics and Its Applications

Assignment Title: Cracking the Code: The Pigeonhole Principle in Hashing

Assignment Type: Case Study

CO: CO1 – Apply combinatorial techniques to solve discrete mathematical problems in computer science.

Bloom's Taxonomy Level: Analyze

Scenario / Context

A hash table maps a large space of possible inputs into a much smaller number of buckets. The Pigeonhole Principle guarantees that once the number of items exceeds the number of buckets, at least one bucket must receive more than one item — a collision. The Generalized Pigeonhole Principle tells you exactly how many items force a bucket to hold a given number of items.

Student Task

You will hash a small dataset into buckets, demonstrate collisions, and use the Pigeonhole Principle to predict the minimum guaranteed collisions before actually hashing.

Instructions / Activity Steps

1. Choose a simple hash function $h(x) = x \bmod n$ for a small n (e.g., $n = 10$), and prepare a dataset of at least 15 distinct items (integers, or ASCII sums of names/words).
2. Compute the hash value of every item and record which bucket (0 to $n-1$) each falls into.
3. Before looking at your actual results, apply the Pigeonhole Principle to determine the minimum number of items guaranteed to collide when placing m items into n buckets.
4. Apply the Generalized Pigeonhole Principle to determine the minimum bucket size guaranteed when m items are placed in n buckets.
5. Compare your theoretical guarantee with the actual collision pattern you observed in your data.

Questions for Analysis

1. How many items would you need to hash into 10 buckets to guarantee at least one bucket has 5 items? Show the generalized pigeonhole calculation.
2. Real hash tables use techniques such as chaining or open addressing to handle collisions — relate one of these to what you observed in your dataset.
3. Why can no hash function avoid collisions entirely once the number of possible inputs exceeds the number of buckets?

Expected Submission

- Hash table with all bucket assignments
- Pigeonhole calculations (basic and generalized)
- Written comparison of predicted vs. observed collisions
- Short note on a collision-handling technique



ICT Tool / Resources Required

Python or MS Excel for hashing simulation

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of the Pigeonhole Principle (basic and generalized)	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct construction of the hash table and bucket assignment	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate pigeonhole calculations and comparison with actual results	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of tables and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original discussion of collision-handling techniques	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 1: Introduction to Combinatorics and Its Applications

Assignment Title: Catching the Copycats: Inclusion-Exclusion in Data Deduplication

Assignment Type: Technology-Based Activity

CO: CO1 – Apply combinatorial techniques to solve discrete mathematical problems in computer science.

Bloom's Taxonomy Level: Evaluate



Scenario / Context

Data-deduplication systems flag a record as a possible duplicate if it matches on any of several overlapping criteria — same email, same phone prefix, similar name. Counting how many records are flagged by at least one rule (without double-counting overlaps) is a job for the Inclusion-Exclusion Principle. A related idea, derangements, models situations such as a shuffled playlist in which no song ends up in its original position.

Student Task

You will model three overlapping duplicate-detection rules over a sample dataset using Inclusion-Exclusion, then separately compute derangements for a shuffled-playlist scenario.

Instructions / Activity Steps

1. Create or collect a sample dataset of about 20 records (real or fictional), and define 3 duplicate-detection rules (e.g., same email domain, same phone area code, similar name spelling).
2. Count how many records satisfy each rule individually, each pairwise overlap, and all three rules together.
3. Apply the Inclusion-Exclusion Principle to compute the total number of records flagged by at least one rule.
4. Separately, take a playlist of 6 songs and compute the number of derangements — shuffles in which no song stays in its original position.
5. Verify your derangement count using the derangement recurrence relation.

Questions for Analysis

1. How did the Inclusion-Exclusion result differ from simply adding the three individual rule-counts? Why does this difference matter for a real deduplication system?
2. For your 6-song playlist, what fraction of all possible shuffles are true derangements? Does this match the known limiting probability of roughly $1/e$ reasonably well?
3. Where else in computing might derangements matter (for example, assigning peer-review tasks so no one reviews their own submitted work)?

Expected Submission

- Dataset with rule tags marked for each record
- Inclusion-Exclusion working and final flagged-record count
- Derangement calculation and recurrence-based verification
- Written comparison of the two techniques

ICT Tool / Resources Required

MS Excel or Python for set counting

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of Inclusion-Exclusion and derangements	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct rule-based tagging and overlap counts	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate application of Inclusion-Exclusion and derangement recurrence	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Presentation / clarity of working	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original comparison and real-world connection	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 1: Introduction to Combinatorics and Its Applications

Assignment Title: Predicting Growth: Recurrence Relations Behind Recursive Algorithms

Assignment Type: AI/Computing Application

CO: CO1 – Apply combinatorial techniques to solve discrete mathematical problems in computer science.

Bloom's Taxonomy Level: Create

Scenario / Context

The running time of many recursive algorithms — naive Fibonacci, recursive linear search, Tower of Hanoi — is described by a recurrence relation. Solving that recurrence, whether by the characteristic-equation method or via generating functions, gives a closed-form formula that predicts exactly how the algorithm's cost grows.

Student Task

You will derive a recurrence relation for a recursive algorithm's step-count, solve it two ways, and



confirm both solutions agree with hand-computed values.

Instructions / Activity Steps

1. Pick a simple recursive algorithm (e.g., naive recursive Fibonacci, recursive linear search, or Tower of Hanoi) and write its step-count recurrence relation $T(n)$ in terms of earlier terms.
2. Compute $T(n)$ by hand or spreadsheet for $n = 1$ to 10 directly from the recurrence.
3. Solve the recurrence using the characteristic equation method (or repeated substitution) to obtain a closed-form expression.
4. Set up the ordinary generating function for the sequence and use it to derive (or verify) the same closed form.
5. Compare the closed-form prediction with your hand-computed values for $n = 1$ to 10 to confirm correctness.

Questions for Analysis

1. What does the growth rate of $T(n)$ tell you about the algorithm's practical usability for large n ?
2. Where did the generating-function approach agree or disagree with your characteristic-equation solution, and why should the two ultimately match?
3. If you doubled the input size, how much longer would the algorithm take, based on your closed-form solution?

Think Beyond the Formula

- How would you redesign the algorithm (e.g., using memoization) to change its recurrence and drastically reduce this growth rate?

Expected Submission

- Recurrence relation derivation
- Hand-computed table for $n = 1$ to 10
- Closed-form solution via the characteristic equation
- Generating function derivation
- Comparison table confirming both methods agree

ICT Tool / Resources Required

MS Excel or MATLAB Octave for verification

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of recurrence relations and generating functions	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct derivation of the	Full marks	Mostly	Partially	Incomplete or	3

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
recurrence and closed-form solution	— thorough, accurate & insightful	complete & correct, minor gaps	complete, basic understanding shown	largely incorrect	
Accurate generating-function derivation and cross-verification	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of working and tables	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original discussion of algorithmic improvement (e.g., memoization)	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 2: Numerical Methods

Assignment Title: Root of the Problem: Finding Break-Even Points

Assignment Type: Data Analysis Task

CO: CO2 – Analyze numerical errors and implement root-finding methods, linear system solvers, and interpolation techniques to solve mathematical problems effectively.

Bloom's Taxonomy Level: Apply

Scenario / Context

The break-even point, where cost equals revenue, is often the root of a nonlinear equation with no neat closed form. Numerical root-finding methods such as Bisection and Newton-Raphson locate this root iteratively, at different speeds and with different reliability guarantees.

Student Task

You will model a nonlinear cost/revenue (or any transcendental) equation and find its root using both the Bisection Method and the Newton-Raphson Method, comparing their convergence.

Instructions / Activity Steps

1. Define a nonlinear function $f(x)$ representing (Revenue – Cost) for a hypothetical product, or choose any transcendental equation of interest.
2. Identify an interval $[a, b]$ where a root exists by checking that $f(a)$ and $f(b)$ have opposite signs.
3. Apply the Bisection Method for at least 6 iterations, recording the interval, midpoint, and $f(\text{midpoint})$ at each step.



4. Apply the Newton-Raphson Method from a suitable starting point on the same equation, recording x_n , $f(x_n)$, and $f'(x_n)$ each iteration until convergence.
5. Compare how many iterations each method needed to reach a similar level of accuracy.

Questions for Analysis

1. Which method converged faster, and by roughly what factor? Relate this to the theoretical convergence order of each method.
2. What would happen to Newton-Raphson if you chose a poor starting point (e.g., near a point where $f'(x) = 0$)? Try it and report what you observe.
3. Why might a real business dashboard prefer the slower-but-more-reliable Bisection Method over the faster Newton-Raphson for automated break-even calculations?

Expected Submission

- Iteration tables for both methods
- Final root value obtained by each method
- Convergence comparison (iteration count, accuracy)
- Written recommendation with justification

ICT Tool / Resources Required

MS Excel or MATLAB

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of root-finding methods and convergence	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct construction and execution of both iteration methods	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate convergence comparison and analysis	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of iteration tables and presentation	Full marks — thorough, accurate &	Mostly complete & correct, minor	Partially complete, basic understanding	Incomplete or largely incorrect	2



Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
	insightful	gaps	shown		
Original recommendation with sound reasoning	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 2: Numerical Methods

Assignment Title: Solving the Network Load-Balancing Puzzle

Assignment Type: Case Study

CO: CO2 – Analyze numerical errors and implement root-finding methods, linear system solvers, and interpolation techniques to solve mathematical problems effectively.

Bloom's Taxonomy Level: Analyze

Scenario / Context

Balancing load, current, or flow across a network of connected nodes — servers, circuits, or pipelines — leads to a system of linear equations. Iterative methods such as Gauss-Seidel and Jacobi solve such systems efficiently, especially when they are large and sparse, unlike exact elimination methods.

Student Task

You will formulate a small network-balancing problem as a system of linear equations and solve it using Gauss Elimination (direct) and both Gauss-Seidel and Jacobi (iterative), comparing the results.

Instructions / Activity Steps

1. Design a simple network of 3–4 nodes (servers or circuit nodes) with known connections and constraints, and formulate the balancing conditions as 3–4 simultaneous linear equations.
2. Solve the system exactly using Gauss Elimination (or Gauss-Jordan), showing all row operations.
3. Solve the same system using the Gauss-Seidel iterative method from an initial guess, for at least 5 iterations.
4. Repeat using the Jacobi iterative method for the same number of iterations from the same initial guess.
5. Compare the Gauss-Seidel and Jacobi results against your exact Gauss Elimination answer at each iteration.

Questions for Analysis

1. Which iterative method — Gauss-Seidel or Jacobi — converged closer to the exact answer faster, and why (relate this to how each uses updated values)?
2. Under what diagonal-dominance condition would you trust an iterative method to converge at all?



Check whether your system satisfies it.

3. Why might a real-time load-balancing system prefer an iterative method over exact Gauss Elimination, even though iterative methods only approximate the answer?

Expected Submission

- Linear system formulation from the network scenario
- Gauss Elimination working
- Gauss-Seidel and Jacobi iteration tables
- Comparison and convergence discussion

ICT Tool / Resources Required

MS Excel, Python, or MATLAB

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of direct and iterative linear-system solving methods	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct formulation and Gauss Elimination working	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate Gauss-Seidel and Jacobi iterations with comparison	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of tables and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original discussion of convergence and diagonal dominance	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10



Unit 2: Numerical Methods

Assignment Title: Predicting Missing Data Points: The Interpolation Detective

Assignment Type: Data Analysis Task

CO: CO2 – Analyze numerical errors and implement root-finding methods, linear system solvers, and interpolation techniques to solve mathematical problems effectively.

Bloom's Taxonomy Level: Apply

Scenario / Context

Sensor logs, stock prices, and server metrics often have gaps — a reading missing at a particular timestamp. Interpolation methods estimate these missing values from the surrounding known data, with different formulas suited to equally spaced versus unevenly spaced data.

Student Task

You will take a time-series dataset with a few points deliberately hidden, and estimate the missing values using Newton's Forward/Backward Interpolation and Lagrange's Interpolation.

Instructions / Activity Steps

1. Collect or simulate at least 8 equally spaced data points (e.g., hourly temperature, CPU usage, or stock closing price) and hide 2 of them to interpolate later.
2. Using the remaining data, apply Newton's Forward Interpolation formula to estimate a missing value near the start of the range.
3. Apply Newton's Backward Interpolation formula to estimate a missing value near the end of the range.
4. Apply Lagrange's Interpolation formula (which does not require equal spacing) to estimate one of the same missing values, and compare.
5. Compare your interpolated values with the actual hidden values and compute the percentage error for each method.

Questions for Analysis

1. Which method gave you the smallest error for your dataset, and does this match what you would expect given where in the range the missing point was located?
2. Lagrange's Interpolation doesn't require equally spaced data — what real-world data situation would make this an essential advantage over Newton's method?
3. What happens to interpolation accuracy if the missing point is far from all known data points (extrapolation) rather than between them?

Expected Submission

- Dataset table with hidden and known values marked
- Newton's Forward, Backward, and Lagrange's interpolation workings
- Error comparison table
- Written conclusion



ICT Tool / Resources Required

MS Excel or Python; any free time-series dataset source

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of interpolation concepts and formulas	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct application of Newton's Forward/Backward Interpolation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Correct application of Lagrange's Interpolation and error comparison	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of dataset presentation and tables	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original conclusion connecting to real data scenarios	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 2: Numerical Methods

Assignment Title: Which Method Wins? A Numerical Methods Showdown

Assignment Type: AI/Computing Application

CO: CO2 – Analyze numerical errors and implement root-finding methods, linear system solvers, and interpolation techniques to solve mathematical problems effectively.

Bloom's Taxonomy Level: Evaluate

Scenario / Context

Every numerical method trades off speed, reliability, and information requirements differently.



Choosing the right method for a computational tool means evaluating multiple candidates on the very same problem, not just picking one by habit.

Student Task

You will solve the same nonlinear equation using Bisection, Regula-Falsi, Newton-Raphson, and Secant methods, and evaluate which is best suited to different practical constraints.

Instructions / Activity Steps

1. Choose one nonlinear equation $f(x) = 0$ with a known or verifiable root (e.g., a cubic or transcendental equation).
2. Solve it using all four methods — Bisection, Regula-Falsi, Newton-Raphson, and Secant — recording iteration count and final error for each, using the same stopping tolerance.
3. Tabulate iteration count, whether derivative information was required, and final accuracy for each method.
4. Rank the four methods for a scenario where computing $f'(x)$ is expensive or unavailable.
5. Rank the four methods again for a scenario where guaranteed convergence matters more than speed (e.g., a safety-critical embedded system).

Questions for Analysis

1. Which method required the fewest iterations overall, and what made it faster (using derivative information, or a better search strategy)?
2. Under what condition could Newton-Raphson fail to converge at all, and which of the other three methods would still work reliably in that case?
3. If you were building a general-purpose root-finding library function used by many other programmers, which method(s) would you choose as defaults, and why?

Expected Submission

- Full iteration tables for all four methods
- Comparison table (iterations / derivative-required / accuracy)
- Two ranked recommendations with justification

ICT Tool / Resources Required

Python, MATLAB or MS Excel

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of all four root-finding methods	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Correct execution of Bisection, Regula-Falsi, Newton-Raphson and Secant	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Analysis of trade-offs (speed, reliability, derivative requirement)	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of comparison table and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original, well-justified recommendations	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 3: Operations Research and Linear Programming

Assignment Title: Maximizing Profit: The Graphical LPP Challenge

Assignment Type: Real-Life Investigation

CO: CO3 – Formulate and solve problems in Linear Programming, Transportation, Assignment, and Game Theory, and apply them to real-life decision-making.

Bloom's Taxonomy Level: Apply

Scenario / Context

A small business — a bakery, print shop, or app-development team — must allocate limited resources such as time, budget, or raw materials between two products or services to maximize profit. With just two products, this is a classic two-variable Linear Programming Problem that can be solved graphically.

Student Task

You will formulate a realistic two-product resource-allocation scenario as an LPP and solve it graphically.

Instructions / Activity Steps



1. Choose a realistic scenario with two products/services and at least two limited resources (e.g., machine hours and raw material), noting or reasonably assuming profit per unit and resource usage per unit.
2. Formulate the objective function (maximize profit) and the constraints (resource limits, non-negativity) as inequalities.
3. Plot all constraints on a graph and shade the feasible region.
4. Identify all corner points of the feasible region and evaluate the objective function at each.
5. Identify the optimal solution (maximum profit) and the corresponding production quantities.

Questions for Analysis

1. Which resource constraint is “binding” (fully used) at your optimal solution, and which has slack (leftover capacity)?
2. If the profit per unit of one product increased, at what point would the optimal corner point change? Try changing the objective function's slope and observe.
3. What would an “unbounded” or “infeasible” LPP look like graphically, and under what business condition might that occur?

Expected Submission

- LPP formulation (objective function and constraints)
- Graph with feasible region and corner points marked
- Corner-point evaluation table
- Optimal solution with interpretation

ICT Tool / Resources Required

GeoGebra, Desmos, or graph paper

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of LPP formulation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct graphing of constraints and feasible region	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate corner-point evaluation and optimal solution	Full marks — thorough, accurate &	Mostly complete & correct, minor	Partially complete, basic understanding	Incomplete or largely incorrect	2

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
	insightful	gaps	shown		
Clarity of graph and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original interpretation of the business result	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 3: Operations Research and Linear Programming

Assignment Title: From Warehouse to Customer: Solving the Transportation Puzzle

Assignment Type: Case Study

CO: CO3 – Formulate and solve problems in Linear Programming, Transportation, Assignment, and Game Theory, and apply them to real-life decision-making.

Bloom's Taxonomy Level: Analyze

Scenario / Context

Logistics companies must ship goods from multiple warehouses to multiple destinations at minimum total cost. This is the classic Transportation Problem: the North-West Corner Rule and Least Cost Method give an initial feasible plan, and the MODI method refines it to the optimal one.

Student Task

You will design a 3×3 transportation scenario (3 sources, 3 destinations) with supply, demand, and shipping costs, and solve it to optimality.

Instructions / Activity Steps

1. Design a transportation table with 3 sources (warehouses) and 3 destinations (stores/cities), each with a supply/demand figure, and assign realistic per-unit shipping costs for every source-destination pair, ensuring total supply equals total demand.
2. Find an Initial Basic Feasible Solution (IBFS) using the North-West Corner Rule.
3. Find a second IBFS for the same table using the Least Cost Method, and compare its total cost with the North-West Corner solution.
4. Starting from your Least Cost Method solution, apply the MODI method to test and achieve optimality.
5. Report the final optimal shipping plan and its total cost.



Questions for Analysis

1. How much more expensive was the North-West Corner IBFS compared to the Least Cost Method IBFS, and why does ignoring cost during allocation lead to this gap?
2. What does a negative opportunity cost in the MODI method tell you to do next?
3. What would “degeneracy” look like in your transportation table, and how would you handle it if it occurred?

Expected Submission

- Transportation table with supply, demand, and costs
- North-West Corner IBFS and its total cost
- Least Cost Method IBFS and its total cost
- MODI optimality-test working
- Final optimal plan and cost, with written comparison

ICT Tool / Resources Required

MS Excel; Python or manual working

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of the Transportation Problem	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct North-West Corner and Least Cost IBFS construction	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate MODI optimality testing and final solution	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of tables and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original comparison and discussion of degeneracy	Full marks —	Mostly complete	Partially complete,	Incomplete or largely	1



Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
	thorough, accurate & insightful	& correct, minor gaps	basic understanding shown	incorrect	
Total Marks					10

Unit 3: Operations Research and Linear Programming

Assignment Title: Who Does What Best? The Assignment Problem

Assignment Type: Technology-Based Activity

CO: CO3 – Formulate and solve problems in Linear Programming, Transportation, Assignment, and Game Theory, and apply them to real-life decision-making.

Bloom's Taxonomy Level: Analyze

Scenario / Context

Assigning tasks or projects to team members — or servers to jobs, employees to shifts — where each pairing has a different cost or time, is solved optimally using the Hungarian Method rather than trial and error.

Student Task

You will design a 4×4 cost/time matrix for assigning 4 people (or machines) to 4 tasks, and solve it using the Hungarian Method.

Instructions / Activity Steps

1. Create a 4×4 matrix showing the time (or cost) each of 4 people/machines would take to complete each of 4 tasks (real estimates from your class/team, or realistic fictional data).
2. Apply row reduction (subtract each row's minimum from every entry in that row) and column reduction (subtract each column's minimum from every entry in that column).
3. Cover all zeros using the minimum possible number of lines; if fewer than 4 lines are needed, adjust the matrix and repeat until an optimal assignment becomes possible.
4. Identify the optimal one-to-one assignment (each person to exactly one task) with the minimum total cost/time.
5. Verify your solution by checking it against at least one alternative valid assignment to confirm it is indeed minimal.

Questions for Analysis

1. How much time/cost would be wasted if tasks were assigned randomly instead of using the Hungarian Method? Compare with a random assignment.
2. Why must the number of people equal the number of tasks for the standard Hungarian Method to apply directly — what would you do with unequal numbers?
3. Where else could this same assignment logic be applied in computing (e.g., assigning jobs to



servers, or matching resumes to job openings)?

Expected Submission

- Cost/time matrix
- Row and column reduction working
- Line-covering steps and adjustments
- Final optimal assignment
- Comparison with a random assignment

ICT Tool / Resources Required

MS Excel or manual working

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of the Assignment Problem and Hungarian Method	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct row/column reduction and line-covering steps	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate final optimal assignment	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of working and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original comparison with random assignment	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10



**SILVER OAK
UNIVERSITY**
EDUCATION TO INNOVATION



Unit 3: Operations Research and Linear Programming

Assignment Title: Game On: Strategic Decisions Using Game Theory

Assignment Type: Real-Life Investigation

CO: CO3 – Formulate and solve problems in Linear Programming, Transportation, Assignment, and Game Theory, and apply them to real-life decision-making.

Bloom's Taxonomy Level: Evaluate

Scenario / Context

Two competing companies — for instance, two food-delivery apps deciding on discount strategies — can be modeled as a two-person zero-sum game, where one player's gain is exactly the other's loss.

Student Task

You will model a realistic competitive business scenario as a two-person zero-sum game payoff matrix and solve it for the optimal strategy.

Instructions / Activity Steps

1. Choose two competing entities and 2–3 strategies each could adopt (e.g., “discount”, “no discount”, “loyalty points”), and construct a payoff matrix showing Player A's gain/loss under every strategy combination.
2. Check for a saddle point using the maximin (row) and minimax (column) criteria.
3. If a saddle point exists, identify the pure-strategy solution and the value of the game.
4. If no saddle point exists, apply the dominance principle to reduce the matrix, then solve for mixed strategies using the algebraic method.
5. Interpret the game's value in terms of the real-world scenario you modeled.

Questions for Analysis

1. Does your game have a saddle point? What does this tell you about whether either player should ever deviate from their single best strategy?
2. If no saddle point existed, what do the mixed-strategy probabilities tell each player about how often to use each strategy?
3. In real business competition, why might companies not behave exactly as this zero-sum model predicts?

Expected Submission

- Payoff matrix
- Saddle-point check (maximin/minimax)
- Dominance reduction (if applicable)
- Mixed-strategy solution (if applicable)
- Game value and real-world interpretation



ICT Tool / Resources Required

MS Excel or manual working

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of zero-sum games and saddle points	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct construction of the payoff matrix	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate saddle-point/dominance/mixed-strategy solving	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of working and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original real-world interpretation of the game's value	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 4: Lattices and Boolean Algebra

Assignment Title: Ordering the Chaos: POSETs in File Permission Systems

Assignment Type: Case Study

CO: CO4 – Analyze the structure and properties of lattices and Boolean algebras, and apply these algebraic concepts to logical systems and digital circuits.



Bloom's Taxonomy Level: Apply

Scenario / Context

File and folder permission systems (read < read-write < read-write-execute) or role hierarchies (Viewer < Editor < Admin) form a partial order — not every pair of permissions is comparable, which is exactly what a Partially Ordered Set (POSET) models.

Student Task

You will model a real permission/role hierarchy as a POSET and draw its Hasse Diagram.

Instructions / Activity Steps

1. List at least 6 permission levels or roles from a real system you use (an OS, a cloud app, or an organizational hierarchy), and define the relation “ $\leq$ ” as “is included in the access of”.
2. Determine, for every pair of elements, whether one is comparable to ($\leq$) the other, or whether they are incomparable.
3. Verify the relation is reflexive, antisymmetric and transitive, confirming it is a valid partial order.
4. Draw the Hasse Diagram, removing all relations implied by transitivity and all self-loops.
5. Identify the maximal element(s), minimal element(s), and any incomparable pair(s) in your diagram.

Questions for Analysis

1. Which two roles/permissions in your system are incomparable, and what does that mean practically for the two users holding them?
2. Is your POSET also a Lattice — does every pair of elements have a unique least upper bound and greatest lower bound? Check at least two pairs.
3. Why is antisymmetry (not symmetry) the defining property that separates a POSET's order relation from an ordinary equivalence relation?

Expected Submission

- List of permission/role elements with the defined relation
- Pairwise comparability table
- POSET property verification (reflexive, antisymmetric, transitive)
- Hasse Diagram
- Maximal/minimal element identification

ICT Tool / Resources Required

Any diagramming tool (PowerPoint, Google Drawings, draw.io) or neatly hand-drawn diagram

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of POSETs	Full marks	Mostly	Partially	Incomplete or	2

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
and their properties	— thorough, accurate & insightful	complete & correct, minor gaps	complete, basic understanding shown	largely incorrect	
Correct construction of the relation and comparability table	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3
Accurate Hasse Diagram and maximal/minimal identification	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of diagram and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original discussion connecting to real access-control systems	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 4: Lattices and Boolean Algebra

Assignment Title: Designing Digital Logic: Boolean Algebra in Circuit Design

Assignment Type: Technology-Based Activity

CO: CO4 – Analyze the structure and properties of lattices and Boolean algebras, and apply these algebraic concepts to logical systems and digital circuits.

Bloom's Taxonomy Level: Analyze

Scenario / Context

Every digital circuit — from a simple light-switch system to a CPU's ALU — is governed by Boolean Algebra. Minimizing a Boolean expression using its laws directly reduces the number of logic gates a circuit needs.

Student Task

You will take a real decision-based system with binary inputs, construct its Boolean expression and



truth table, minimize the expression, and verify the minimized circuit.

Instructions / Activity Steps

1. Describe a real decision-making system with 3–4 binary inputs (e.g., sensors, switches, conditions) and one binary output, and write its Boolean expression.
2. Construct the complete truth table for all input combinations and the resulting output.
3. Minimize the Boolean expression using Boolean Algebra laws (De Morgan's, absorption, distributive, etc.), showing each simplification step.
4. Draw the logic circuit diagram (AND/OR/NOT gates) for both the original and the minimized expression, and count the gates used in each.
5. Verify that the minimized expression produces exactly the same truth table as the original.

Questions for Analysis

1. How many logic gates did you save by minimizing the expression, and why does this matter for real hardware (cost, power, speed)?
2. Which Boolean law (e.g., absorption, De Morgan's) contributed most to the simplification in your specific case?
3. Could you have reached the same minimized form using a Karnaugh Map instead of algebraic laws? Briefly describe how you would set one up for your expression.

Expected Submission

- Original Boolean expression and truth table
- Step-by-step minimization
- Both circuit diagrams (original and minimized) with gate counts
- Verification table

ICT Tool / Resources Required

Logic.ly or any free logic-circuit simulator, or hand-drawn diagrams

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of Boolean Algebra laws and minimization	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct truth table construction	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	3

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Accurate step-by-step minimization and verification	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of circuit diagrams and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original discussion of gate savings / Karnaugh Map link	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Unit 4: Lattices and Boolean Algebra

Assignment Title: The Lattice Behind Your Access Control System

Assignment Type: AI/Computing Application

CO: CO4 – Analyze the structure and properties of lattices and Boolean algebras, and apply these algebraic concepts to logical systems and digital circuits.

Bloom's Taxonomy Level: Create

Scenario / Context

Multi-level security systems — such as classification levels Unclassified < Confidential < Secret < Top Secret, combined with department tags — are modeled using lattices, where information may only “flow” from a lower level to a higher or equal one. Distributive and complemented lattice properties determine when clean information-flow rules can be designed.

Student Task

You will design a small multi-level access-control system as a lattice, verify its lattice and Boolean-algebra-relevant properties, and propose an information-flow rule based on it.

Instructions / Activity Steps

1. Design a security/access-control system with at least 4 security levels or label combinations (e.g., combining a clearance level with a department tag, such as {Secret, Engineering}), and define the partial order “ $\leq$ ” as “may flow into”.
2. For every pair of elements, determine the Least Upper Bound (join, $\vee$) and Greatest Lower Bound



(meet, $\wedge$), confirming your structure is a valid Lattice.

3. Check whether your lattice is distributive (does $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ for representative triples?) and whether it is complemented (does every element have a complement satisfying $a \vee a' = 1$ and $a \wedge a' = 0$?).
4. If your lattice qualifies, identify it as (or relate it to) a Boolean Algebra, and identify its atoms (minimal non-zero elements).
5. Propose one information-flow rule for your system and justify it using your lattice's structure.

Questions for Analysis

1. Give one real pair of security labels in your system where computing the join ($\vee$) represents “the minimum access level required to legitimately see both pieces of information.”
2. Is your lattice complemented? If yes, what does the complement of a security label mean in practical security terms?
3. Why would a security system designer care whether their access-control structure forms a distributive lattice rather than just any POSET?

Think Beyond the Formula

- How does this lattice-based thinking relate to real-world “least privilege” and “need-to-know” security principles used in operating systems and databases?

Expected Submission

- Lattice element list with the defined partial order
- Join/meet table for at least 4 pairs
- Distributivity and complementation checks
- Atom identification
- Proposed information-flow rule with justification

ICT Tool / Resources Required

Any diagramming tool for the lattice (Hasse) diagram, or hand-drawn

Rubrics for Evaluation

Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
Understanding of lattices, distributivity and complementation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Correct construction of the lattice and join/meet table	Full marks — thorough, accurate	Mostly complete & correct,	Partially complete, basic understanding	Incomplete or largely incorrect	3



Criteria	Excellent	Good	Satisfactory	Needs Improvement	Marks
	& insightful	minor gaps	shown		
Accurate distributivity/complementation checks and atom identification	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Clarity of diagram and presentation	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	2
Original, well-justified information-flow rule	Full marks — thorough, accurate & insightful	Mostly complete & correct, minor gaps	Partially complete, basic understanding shown	Incomplete or largely incorrect	1
Total Marks					10

Course Coordinator	HoD / HoI
---------------------------	------------------